

Student Management System

DATABASE DESIGN DOCUMENT

MySQL schema · stored procedures · triggers · views · functions · indexes

Engine: MySQL 8.0+ | Normal form: 3NF | Tables: 14

1. Overview

This document describes the relational schema behind the Student Management System: how the 14 tables relate to each other, the constraints that keep data consistent, and the stored procedures, triggers, views, and functions that implement business logic at the database layer rather than only in application code. The schema is normalized to third normal form (3NF) — every non-key column depends on the whole primary key and nothing but the key.

1.1 Entity groups

Identity. users, students, faculty — a shared users table handles login for all three roles; students and faculty each extend it with role-specific profile data.

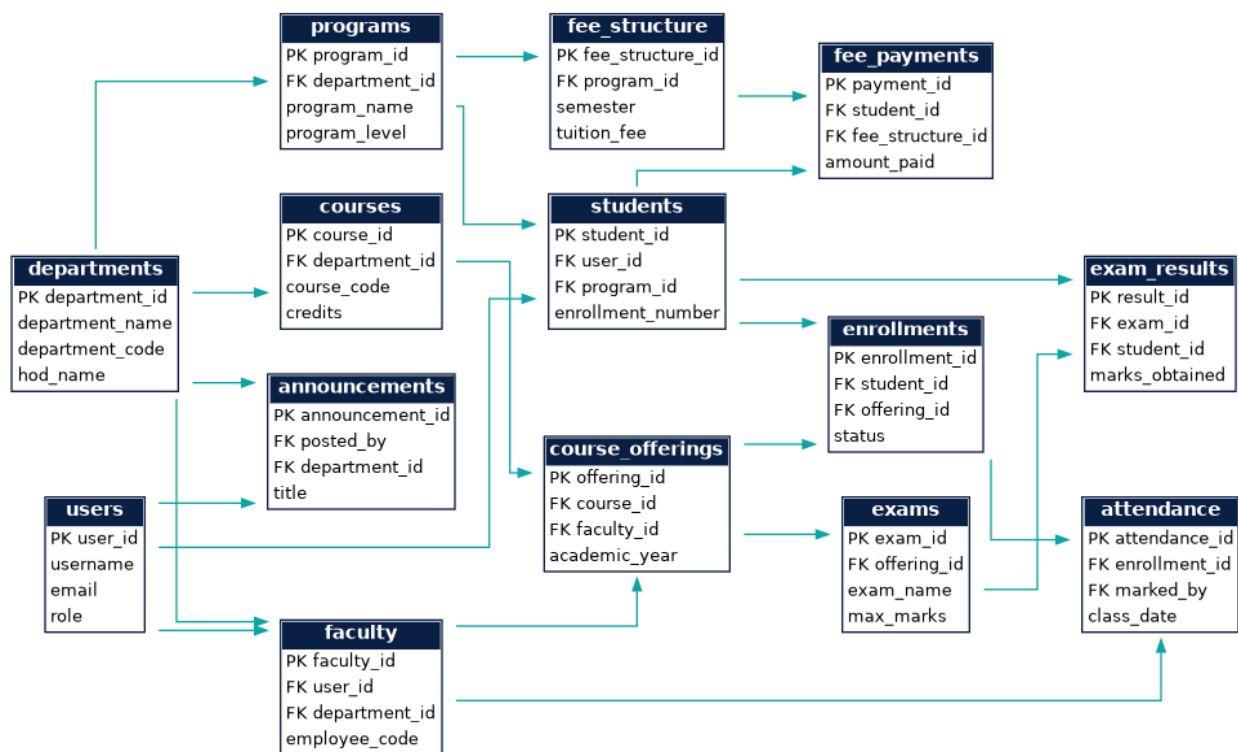
Academic structure. departments, programs, courses, course_offerings — the catalogue and how it's scheduled per term.

Activity. enrollments, attendance, exams, exam_results — what students actually do once enrolled.

Finance. fee_structure, fee_payments — what's owed and what's been paid.

Communication. announcements — notices targeted by role and/or department.

2. Entity-Relationship Diagram



Arrows point from the referenced (parent) table to the referencing (child) table, i.e. from the "one" side to the "many" side of each foreign key relationship.

3. Table Reference

Table	Purpose	Primary Key	Foreign Keys
departments	Top-level academic units.	department_id PK	—
programs	Degree programs offered by a department.	program_id PK	department_id → departments
users	Shared login identity for all three roles.	user_id PK	—
faculty	Faculty profile, one-to-one with a user.	faculty_id PK	user_id → users, department_id → departments
students	Student profile, one-to-one with a user.	student_id PK	user_id → users, program_id → programs
courses	Subject catalogue.	course_id PK	department_id → departments
course_offerings	A course taught by one faculty member in one term.	offering_id PK	course_id → courses, faculty_id → faculty
enrollments	Resolves the student ↔ offering many-to-many.	enrollment_id PK	student_id → students, offering_id → course_offerings
attendance	One row per enrollment per class date.	attendance_id PK	enrollment_id → enrollments, marked_by → faculty
exams	An assessment under one offering.	exam_id PK	offering_id → course_offerings
exam_results	One student's marks on one exam.	result_id PK	exam_id → exams, student_id → students
fee_structure	What's owed, per program/semester/year.	fee_structure_id PK	program_id → programs
fee_payments	A payment against a fee_structure row.	payment_id PK	student_id → students, fee_structure_id → fee_structure
announcements	A notice targeted by role and/or department.	announcement_id PK	posted_by → users, department_id → departments

4. Key Constraints & Design Decisions

4.1 Referential integrity

Most foreign keys use **ON DELETE RESTRICT** for reference data (departments, programs, courses) so a department can't be deleted while programs still point to it. Dependent records that only make sense alongside their parent — a student's own enrollments, a user's own student/faculty profile — use **ON DELETE CASCADE**. `announcements.department_id` uses **ON DELETE SET NULL** since a notice should survive its department being reorganized.

4.2 Uniqueness & business rules enforced as CHECK/UNIQUE constraints

- enrollments: `UNIQUE(student_id, offering_id)` — a student can't double-enroll in the same offering.
- course_offerings: `UNIQUE(course_id, academic_year, semester_term, faculty_id)` — prevents duplicate scheduling.
- attendance: `UNIQUE(enrollment_id, class_date)` — one attendance record per student per day.
- exam_results: `UNIQUE(exam_id, student_id)` — one result per student per exam.
- `students.current_semester` and `courses.semester`: `CHECK BETWEEN 1 AND 12`.
- `exams.weightage_pct`: `CHECK BETWEEN 0 AND 100`.

4.3 Why a shared users table

Admin, faculty, and student accounts all authenticate the same way, so login logic, password hashing, and the JWT role claim live in one place (`users.role`). Faculty and students each get their own profile table joined 1:1 on `user_id`, keeping role-specific columns (e.g. `salary` vs. `guardian_phone`) out of a single bloated table.

5. Stored Procedures

sp_enroll_student

IN student_id, offering_id · OUT status

Wraps the capacity check and INSERT in one transaction with FOR UPDATE row locking, so two students racing for the last seat can't both succeed.

sp_record_fee_payment

IN student_id, fee_structure_id, amount, mode, txn_ref · OUT balance_due

Inserts the payment, then computes and returns the remaining balance in the same transaction.

sp_calculate_final_grade

IN student_id, offering_id

Aggregates weighted exam performance for a course and returns a computed letter grade.

sp_mark_attendance_bulk

IN offering_id, class_date, marked_by, default_status

Marks every actively-enrolled student in one offering for one date in a single transaction (used by the faculty portal's "mark whole class" action).

6. Triggers

Trigger	Fires on	Effect
trg_exam_results_grade	BEFORE INSERT ON exam_results	Auto-computes the letter grade from marks_obtained / max_marks.
trg_exam_results_no_overmark	BEFORE INSERT ON exam_results	Rejects marks_obtained greater than the exam's max_marks.
trg_enrollment_capacity_check	BEFORE INSERT ON enrollments	Second line of defense against over-enrollment beyond the stored procedure.
trg_student_status_on_graduation	BEFORE UPDATE ON students	Auto-flips status to 'graduated' once current_semester exceeds the program's total_semesters.
trg_fee_payment_audit	AFTER INSERT ON fee_payments	Writes an immutable audit row to fee_payment_audit for every completed payment.

7. Views

vw_student_directory. Flattened student + program + department info for admin/faculty listing screens.

vw_course_offering_summary. Enrollment fill-rate per offering (enrolled_count, seats_remaining).

vw_student_attendance_pct. Attendance percentage per student per offering.

vw_student_fee_balance. Outstanding balance per student per fee structure.

vw_faculty_workload. Courses taught and total students, per faculty member.

vw_top_performers. Ranked leaderboard per offering using RANK() OVER (PARTITION BY offering_id ORDER BY weighted_percentage DESC) — a window function over a subquery that computes each student's weighted exam percentage.

8. Functions

fn_letter_grade(pct). Maps a percentage to a letter grade (A+ down to F).

fn_student_age(dob). Current age in years from a date of birth.

fn_attendance_percentage(enrollment_id). Attendance % for one enrollment, computed on read.

fn_outstanding_balance(student_id, fee_structure_id). Remaining fee balance, used by ad-hoc reporting queries.

9. Indexing Strategy

Beyond primary keys and the UNIQUE constraints above, indexes were added for the access patterns the API actually uses: role lookups during auth, status/program filters on the student directory, date-range scans on attendance, and the student/offering lookups that back both directions of the enrollments table. See [database/indexes.sql](#) for the full list.